

C-Bibliotheken einfach erklärt

Wenn du mit **C-Programmierung** anfängst, wirst du früher oder später auf **Bibliotheken** stossen. Aber was genau sind sie und warum brauchen wir sie? Lass uns das Schritt für Schritt anschauen!

Was ist eine Bibliothek in C?

Eine **Bibliothek** ist eine Sammlung von **Funktionen und Code**, die du in mehreren Programmen wiederverwenden kannst. Statt denselben Code immer wieder neu zu schreiben, kannst du einfach eine Bibliothek nutzen.

Es gibt zwei Hauptarten von C-Bibliotheken:

Typ	Beschreibung	Dateiendung
Statische Bibliothek	Wird direkt ins Programm eingebaut	.a (Linux/macOS), .lib (Windows)
Dynamische Bibliothek	Wird zur Laufzeit geladen	.so (Linux/macOS), .dll (Windows)

Wie benutzt man eine Bibliothek?

Wenn du z. B. die **mathematischen Funktionen** (`sqrt()`, `sin()`, `cos()`) nutzen willst, musst du die **mathematische Bibliothek (libm)** einbinden:

```
#include <stdio.h>
#include <math.h>

int main() {
    double x = 9.0;
    printf("Die Wurzel von %.2f ist %.2f\n", x, sqrt(x));
    return 0;
}
```

Beim Kompilieren mit GCC muss man `-lm` angeben, um die Bibliothek einzubinden:

```
gcc main.c -o mein_programm -lm
```

1. Eigene C-Bibliothek erstellen

Jetzt erstellen wir unsere **eigene Bibliothek**!

Header-Datei (zfb.h) – Die öffentliche API

Jede Bibliothek braucht eine **Header-Datei** mit den Funktionsdeklarationen.

Erstelle eine Datei **zfb.h**:

```
#ifndef ZFB_H
#define ZFB_H

// Eine einfache Funktion
void hallo();

#endif
```

Warum #ifndef ZFB_H?

Es verhindert, dass die Datei doppelt geladen wird.

Quellcode (zfb.c) – Die Implementierung

Jetzt schreiben wir den **Code der Bibliothek**. Erstelle eine Datei **zfb.c**:

```
#include <stdio.h>
#include "zfb.h"

void hallo() {
    printf("Hallo aus der ZFB-Bibliothek!\n");
}
```

Bibliothek kompilieren

Jetzt müssen wir aus **zfb.c** eine **Bibliothek** erstellen.

Statische Bibliothek (libzfb.a) erstellen

```
gcc -c zfb.c -o zfb.o  
ar rcs libzfb.a zfb.o
```

Ergebnis: libzfb.a

Dynamische Bibliothek (libzfb.so) erstellen (Linux/macOS)

```
gcc -shared -o libzfb.so zfb.c
```

Ergebnis: libzfb.so

Dynamische Bibliothek (zfb.dll) erstellen (Windows)

```
gcc -shared -o zfb.dll zfb.c
```

Ergebnis: zfb.dll

Eine Bibliothek in einem Programm nutzen

Jetzt erstellen wir ein **Hauptprogramm (main.c)**, das unsere Bibliothek nutzt:

```
#include "zfb.h"

int main() {
    hallo();
    return 0;
}
```

Programm mit der statischen Bibliothek linken

```
gcc main.c -o mein_programm -L. -lzfb
```

Programm mit der dynamischen Bibliothek linken

```
gcc main.c -o mein_programm -L. -Wl,-rpath,. -lzfb
```

Was bedeuten die Flags?

- `-L.` → Sucht nach Bibliotheken im aktuellen Verzeichnis.
- `-lzfb` → Verlinkt die Bibliothek libzfb.a oder libzfb.so.
- `-Wl,-rpath,.` → Sorgt dafür, dass libzfb.so gefunden wird.

Fazit

- **Bibliotheken** sind wiederverwendbarer Code.
- **Statische Bibliotheken (.a, .lib)** werden ins Programm integriert.
- **Dynamische Bibliotheken (.so, .dll)** werden zur Laufzeit geladen.
- **CMake kann helfen**, Bibliotheken plattformunabhängig zu bauen.